



probability for language

LING 430 Computational Linguistics Fall 2026

lecture 6a

Sep 29 2026

Siyu Liang

siyu.liang@rice.edu

note

- no class meeting this week
- use slides and the Python notebook to follow along
 - answers to activities on the slides are at the bottom of the notebook
- feel free to post questions on the Canvas discussion board
- budget about 75 minutes



last time

- evaluation
 - accuracy, confusion matrix, precision & recall, F1
- edit distance, Word Error Rate (WER)



agenda

- probability
- conditional probability
- Bayes' theorem
- estimating probabilities



today's notebook

class06a.ipynb

- course website → week 6 → activity
- **File → Save a copy in Drive**



probability

probability

- a **probability** is a fraction of a finite set
 - survey 1,000 people; 20 are bank tellers; the fraction is 0.02
 - pick one person at random: the probability of a bank teller is 0.02
- count the cases you care about, divide by all the cases
- two rules that follow
 - a probability is between 0 and 1
 - the probabilities of all the outcomes add up to 1

probability

- `nltk's FreqDist` counts the occurrences of strings in a list
- in *Emma*, 4,844 of 192,427 tokens are *the*
 - the probability of *the* in *Emma* is $P(\textit{the}) = 0.025$

```
fd = nltk.FreqDist(tokens)
```

```
fd["the"], fd.N()
```

```
→ (4844, 192427)
```

```
fd.most_common(3)
```

```
→ [(',', 11454), ('.', 6928), ('to', 5183)]
```

```
fd.freq("the")
```

```
→ 0.02517
```

probability

how do we get the probability?

- sometimes from reasoning
 - a fair coin: $P(\text{heads}) = 0.5$
 - a die: $P(\text{1}) = 1/6$, $P(\text{not 1}) = 5/6$
 - we know every outcome, and symmetry makes them equally likely
- usually from experience
 - $P(\text{rain in Houston tomorrow})$: how often it rained on days like tomorrow
 - $P(\text{the next word is } \textit{the})$: the same estimate, from a corpus instead of a weather record
- more data, better estimate; always ask how much data a number was counted over

probability

Linda the banker

- Linda is 31, single, outspoken, very bright, majored in philosophy, and cares deeply about discrimination and social justice; which is more probable?
 - (a) Linda is a bank teller
 - (b) Linda is a bank teller and is active in the feminist movement
- most people choose (b); (b) cannot be more probable
 - every feminist bank teller is a bank teller
 - 1,000 people fit the description and 10 are bank tellers: at most all 10 are also feminists, never more
- “both A and B” can never be more probable than A alone

probability

conjunction

- **conjunction**, $P(A \text{ and } B)$, is the fraction of cases where both hold
 - the same whichever way you write it: $P(A \text{ and } B) = P(B \text{ and } A)$
- in *Emma*: A is “the token is so”, B is “the next token is well”
 - `nltk.bigrams()` pairs every token with the next one; 924 tokens are so, 30 of them followed by well
 - $P(A \text{ and } B) = 30 / 192,426 = 0.000156$
- two conditions on top of each other leave very few cases

```
bigrams = nltk.FreqDist(nltk.bigrams(tokens))
```

```
bigrams[("so", "well")]
```

```
→ 30
```

```
bigrams.freq(("so", "well"))
```

```
→ 0.000156
```

probability

independence

- two events are **independent** when knowing one says nothing about the other
 - two dice: $P(\text{1} \text{ then } \text{1}) = 1/6 \times 1/6 = 1/36$; the first roll does not remember
 - only for independent events is $P(A \text{ and } B) = P(A) \times P(B)$
- language is the opposite of dice
 - $P(\textit{well})$ over the whole novel: 0.0016
 - $P(\textit{well})$ right after *so*: 0.032, twenty times higher
- every word changes the odds of the next one; that dependence is what we model from here on



conditional probability

conditional probability

- **conditional probability** $P(A \mid B)$, “A given B”: of the cases where B holds, the fraction where A also holds
 - restrict the world to B, then count A inside it
 - $P(A \mid B) = P(A \text{ and } B) / P(B)$, Downey’s Theorem 1
- $P(\textit{well} \mid \textit{so})$: keep the 924 tokens that are *so*, count the ones followed by *well*
 - $30 / 924 = 0.032$
- the bar changes the denominator: the whole novel for $P(\textit{well})$, the 924 cases of *so* for $P(\textit{well} \mid \textit{so})$

conditional probability

- `ConditionalFreqDist` is one `FreqDist` per condition, here one per preceding word
- `cf["so"]` is the world after so; `.freq()` inside it is the conditional probability
- the same object answers what the spell checker could not: what usually comes next

```
cf = nltk.ConditionalFreqDist(nltk.bigrams(tokens))
```

```
cf["so"].freq("well")
```

→ 0.0325

```
cf["so"].most_common(4)
```

→ [('much', 96), ('very', 79), (',', 31), ('well', 30)]

conditional probability

- conditional probabilities are **not commutative**
- $P(A \mid B)$ and $P(B \mid A)$ are different questions with different denominators
 - Downey's survey: 77% of bankers are women; 2% of women are bankers
- in *Emma*: Miss 592 times, Woodhouse 313 times, Miss Woodhouse 173 times
 - $P(\text{Woodhouse} \mid \text{after Miss}) = 173 / 592 = 0.29$
 - $P(\text{Miss} \mid \text{before Woodhouse}) = 173 / 313 = 0.55$
- the same 173 pairs, two probabilities: most Woodhouses carry the title, most Misses belong to someone else
- confusing the two directions is the most common error in this area, and the reason Bayes' theorem exists

activity 1

- the notebook prints the counts for *Miss*, *Fairfax* and *Miss Fairfax*
- by hand first, then check with the cell
 - $P(\textit{Fairfax} \mid \textit{after Miss})$
 - $P(\textit{Miss} \mid \textit{before Fairfax})$
- then pick a word of your own and run `cfid[word].most_common(5)`
- which of your two probabilities is larger, and why does the novel make it so?



Bayes' theorem

Bayes' theorem

- an email contains *winner*: does it go to the junk folder?
 - what you want: $P(\text{spam} \mid \text{winner})$
 - what you can count: open the spam folder and read off $P(\text{winner} \mid \text{spam})$
- the same asymmetry everywhere
 - a test result is easy to count among the sick, hard to read for one patient
 - a word is easy to count in a genre, hard to use as evidence of the genre
- **Bayes' theorem** turns the direction you can count into the direction you want

Bayes' theorem

- Theorem 1, the definition: $P(A | B) = P(A \text{ and } B) / P(B)$
- Theorem 2, multiply through: $P(A \text{ and } B) = P(B) \times P(A | B)$
 - a joint probability is a chain: first B, then A given B
- Theorem 3, **Bayes' theorem**: the conjunction is the same both ways, so $P(B) \times P(A | B) = P(A) \times P(B | A)$
 - $P(A | B) = P(A) \times P(B | A) / P(B)$

Bayes' theorem

- $P(A)$: the **prior**, how common A is before any evidence
- $P(B \mid A)$: the **likelihood**, how often the evidence shows up when A holds; the direction you can count
- $P(B)$: how often the evidence shows up at all
- $P(A \mid B)$: the **posterior**, what you believe about A after seeing B
- start from how common A is, then scale by how much more often the evidence appears when A holds

Bayes' theorem

- 1% of people have the disease; the test catches 80% of true cases; it is positive for 9.6% of healthy people
- you test positive; write down your gut $P(\text{disease} \mid \text{positive})$ before reading on
- prior $\times$ likelihood: $0.01 \times 0.8 = 0.008$
- $P(\text{positive})$, every road to a positive result: $0.01 \times 0.8 + 0.99 \times 0.096 = 0.10304$
 - the law of total probability: add up the mutually exclusive ways B can happen
- posterior: $0.008 / 0.10304 = 0.078$, under 8%

Bayes' theorem

- let's scale up the previous example to 10,000 people
- of 1,035 positives, 80 are real: $80 / 1,035 = 0.077$
- this is a confusion matrix, and $P(\text{disease} \mid \text{positive})$ is the test's precision
- the test is good; the prior is what makes a positive result weak evidence

	has it	does not	total
tests positive	80	955	1,035
tests negative	20	8,945	8,965
total	100	9,900	10,000

Bayes' theorem

in Emma

- $P(\text{Miss} \mid \text{before Woodhouse}) = P(\text{Miss}) \times P(\text{Woodhouse} \mid \text{after Miss}) / P(\text{Woodhouse})$
 - $= 0.00308 \times 0.2922 / 0.00163 = 0.55$
 - the same as counting directly

```
p_miss, p_wood = fd.freq("Miss"), fd.freq("Woodhouse")
```

```
p_wood_given_miss = cfd["Miss"].freq("Woodhouse")
```

```
p_miss * p_wood_given_miss / p_wood
```

→ 0.5527

activity 2

- suppose you have a spam folder
 - 20% of all email is spam
 - *winner* appears in 5% of spam and in 0.1% of legitimate mail
- by hand in the notebook: $P(\text{spam} \mid \text{winner})$
 - you can use the medical example as the template
- then change one number: spam is 60% of all email instead of 20%
- which input mattered most, and does that match your intuition about spam filters?



estimating probabilities

estimation

- a count is an estimate
- 192,427 tokens of one novel are a sample of English, not English
- $P(\textit{the}) = 0.025$ is the best guess the sample allows
- a probability from ten observations and one from ten thousand look identical on paper; only the second deserves trust
- a word that never occurs in the sample gets probability 0, and that is wrong
 - *Emma* has no *computer*; English does
 - the problems of 0s will be covered on Thursday

estimation

maximum likelihood

- a coin of unknown bias comes up H H H T T
 - we call $P(\text{heads})$ θ
 - theta θ is used in place of an unknown parameter
- flips are independent, so $P(\text{H H H T T}) = \theta \times \theta \times \theta \times (1 - \theta) \times (1 - \theta)$
 - $\theta = 0.5$ gives 0.03125; $\theta = 0.6$ gives 0.03456
- **maximum likelihood**: choose the value under which the observed data is most probable

```
for t in [0.5, 0.6, 0.7]:  
    ...     print(t, round(t**3 * (1 - t)**2, 5))
```

→ 0.5 0.03125

→ 0.6 0.03456

→ 0.7 0.03087

estimation

- the winner is $\theta = 0.6$, which is 3 heads over 5 flips
- this is not a coincidence
 - for counts, the maximum likelihood estimate is always the relative frequency
- so “training on a corpus” means counting and normalizing
 - maximum likelihood is the argument that those relative frequencies are the right numbers
- it says nothing about the words you never saw
 - the zero problem

estimation

- a sentence probability is a product of many numbers below 1, and it shrinks fast
 - a computer cannot hold 0.1 to the power 1,000; it comes out as 0
- logarithms turn products into sums: $\log(a \times b) = \log(a) + \log(b)$
- probabilities are stored and added as **log probabilities**
 - the largest log probability is still the largest probability

```
0.1 ** 1000
```

→ 0.0

```
sum(log(0.1) for i in range(1000))
```

→ -2302.585092994075



next time

- Thursday, asynchronous
 - n-gram language models
- **HW 3 due Thursday, Oct 1, 11:59 pm**
- in-person classes resume Tuesday, Oct 6